



YAZILIM MÜHENDİSLİĞİ FİNAL PROJE RAPORU

Proje Adı: DevSözlük - Yazılımcılar İçin Teknik Forum Platformu

Grup Numarası: Grup 7

GitHub Repository Linki:

<https://github.com/tevfikgorel5/Yazilimci-Birlestirme-Platformu-Projesi>

Tarih: 19 Mayıs 2026

Proje Ekibi:

- Tevfik Görel - 25100011407
- Sefa Koyuncu - 24100011023
- Mert Sefa Taş - 25100011416
- Hamza Ergüven - 23100011071

İÇİNDEKİLER

1. Giriş
2. Proje Deneyimi
3. Yapay Zeka Kullanımı
4. Kullanıcı Kılavuzu
5. Kurulum Talimatları
6. Görev Dağılımı
7. Yaptıklarımız, Yapmadıklarımız ve Nedenleri

1. GİRİŞ

DevSözlük, yazılım geliştiricilerin karşılaştıkları teknik sorunları paylaşabildikleri, topluluktan hızlı destek alabildikleri ve çözülen problemlerin geleceğe dönük aranabilir bir bilgi tabanı olarak arşivlendiği modern bir teknik forum platformudur.

Bu rapor, projemizin analiz ve tasarım aşamalarından geçerek ulaştığı nihai durumu, uygulama sürecinde karşılaşılan teknik zorlukları, ekibimizin edindiği yazılım mühendisliği deneyimlerini ve projenin çalışır halinin dökümantasyonunu içermektedir. Projemiz an itibarıyla minimum çalışan ürün (MVP) seviyesine ulaşmış olup; kullanıcı kaydı, yetkilendirme, başlık oluşturma, başlıklar altında içerik (entry) üretme ve yönetici (admin) moderasyonu gibi temel yapıtaşlarını başarılı bir şekilde yerine getirmektedir.

2. PROJE DENEYİMİ

Bu proje, ekibimiz için teorik yazılım mühendisliği prensiplerini gerçeğe dönüştürdüğümüz, analizin ve tasarımın kodlama kalitesini nasıl doğrudan etkilediğini bizzat deneyimlediğimiz bir süreç olmuştur.

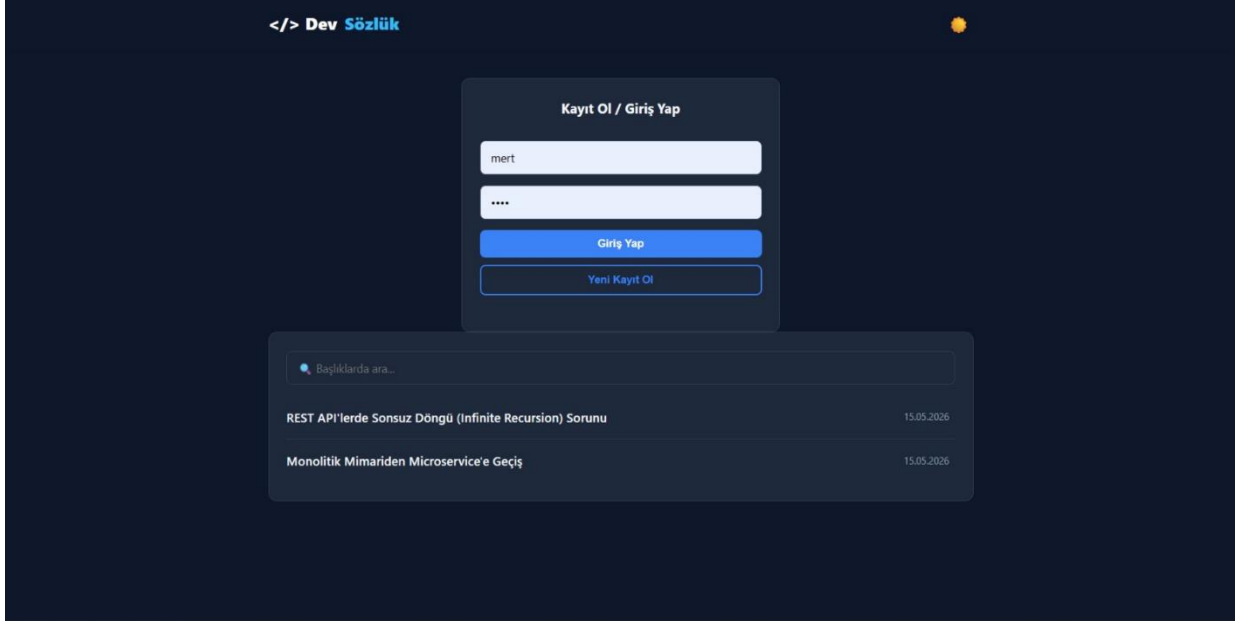
- **Mimari Kararlar ve Öğrenilen Teknolojiler:** Arka yüz mimarisinde kurumsal standartlara uygun, esnek ve güvenilir bir RESTful API altyapısı kurabilmek amacıyla **Java Spring Boot** framework'ünü tercih ettik. Veritabanı yönetiminde, nesne yönelimli programlama (OOP) ile ilişkisel veritabanı (RDBMS) arasındaki köprüyü kuran ORM mantığını öğrendik ve **Spring Data JPA** kütüphanesini kullandık. Veritabanımızı yerelde çalıştırmak yerine bulut tabanlı bir PostgreSQL servisi olan **Neon.tech** üzerinde konumlandırarak modern bulut mimarilerini ve uzak veritabanı bağlantı yönetimini deneyimledik. Ön yüzde ise karmaşık framework'lerin (React/Vue vb.) getireceği ek yüklerden kaçınarak, sistem performansını yüksek tutmak adına saf **HTML5, CSS3 ve Vanilla JavaScript (Fetch API)** kullandık.
- **UML Diyagramlarının Kodlamaya Katkısı:** Proje sürecinde çizdiğimiz **Sınıf (Class) Diyagramı** ve **Sıra (Sequence) Diyagramları**, implementasyon aşamasında bizim için adeta bir yol haritası oldu. Kullanıcı, Baslık ve Cevap varlıkları (Entity) arasındaki 1-to-N (Bire Çok) ilişkileri kurgularken ve veritabanı seviyesindeki ON DELETE CASCADE (Cascade mekanizmasıyla otomatik silme) kurallarını tanımlarken sınıf diyagramımıza birebir sadık kaldık. Tasarım aşamasında doğru planlama yapmanın, kodlama sırasındaki kaybolmaları ve mimari hataları ne kadar büyük ölçüde engellediğini fark ettik.
- **Yazılım Mühendisliği Bakış Açımızın Değişimi:** Bu projeden önce ekibimiz için yazılım geliştirmek çoğunlukla "doğrudan kod editörünü açıp akla gelen ilk mantıkla kod yazmak" demekti. Ancak BİL 204 süreciyle birlikte, kod yazmanın aslında bir projenin sadece son aşaması olduğunu anladık. Kullanım senaryolarını (Use Case) belirlemenin, sistem ön koşullarını analiz etmenin ve aktörlerin (Standart Kullanıcı, Yönetici) yetki sınırlarını net olarak çizmenin projeyi ne kadar sürdürülebilir kıldığını gördük. Yazılıma sadece bir ürün değil, dökümantasyonuyla, mimarisiyle ve test edilebilirliğiyle yaşayan bir "mühendislik sistemi" olarak bakmayı öğrendik.

3. YAPAY ZEKA KULLANIMI

Geliştirme sürecimizde, kodlama hızımızı artırmak ve karşılaştığımız hataları anlamlandırmak için yapay zeka (LLM) araçlarından aktif bir asistan olarak faydalandık. Yapay zekayı bir "kod yazıcı" olarak değil, bir "kıdemli geliştirici (senior developer) / mentör" olarak konumlandırdık.

- **Kullanılan Araçlar:** Geliştirme süreci boyunca ağırlıklı olarak **Gemini** ve **ChatGPT** kullanılmıştır.
- **Kullanıldığı Yerler ve Katkısı:**
 - **Hata Ayıklama (Debugging):** Örneğin, Java'da Controller sınıflarını yazarken aldığımız error: cannot find symbol hatasını yapay zekaya sorduk. Sorunun List sınıfının import edilmemesinden (import java.util.List;) kaynaklandığını açıkladı. Hatayı sadece çözmekle kalmayıp, nedenini de öğrenmiş olduk.
 - **Boilerplate Kod Üretimi:** Frontend kısmında index.html içerisindeki CSS tasarımları ve temel Fetch API isteklerinin iskeletini oluşturmak için AI'dan destek aldık. Ancak dönen kodları doğrudan kullanmadık; kendi belirlediğimiz API endpoint'lerine (/api/baslik/olustur vb.) ve ID parametrelerine göre uyarladık.
 - **SQL ve Veritabanı Yönetimi:** Sistemde test yapabilmek adına bir kullanıcıya admin yetkisi vermemiz gerektiğinde, Neon.tech arayüzünde çalıştırmak üzere gerekli SQL güncelleme komutunu (UPDATE kullanıcı SET rol = 'ADMIN' WHERE id = 1;) yapay zeka yardımıyla oluşturduk.
- **Yapay Zekanın Sınırlamaları ve Hatalı Yönlendirmeleri:**
 - Süreç içerisinde yapay zekanın "aşırı mühendislik" (overengineering) eğiliminde olduğunu fark ettik. Projemiz henüz bir MVP (Minimum Çalışan Ürün) aşamasında olmasına rağmen, AI basit bir kullanıcı kayıt/giriş işlemi için doğrudan Spring Security ve JWT (JSON Web Token) gibi projenin kapsamını aşan, çok karmaşık kurumsal konfigürasyonlar önerdi. Yapay zekanın bu yönlendirmelerine kapılmayıp, projemizin mevcut ölçeğine ve gereksinimlerine uygun, daha sade ve bizim kontrolümüzde olan temel bir yetkilendirme mimarisine sadık kaldık.

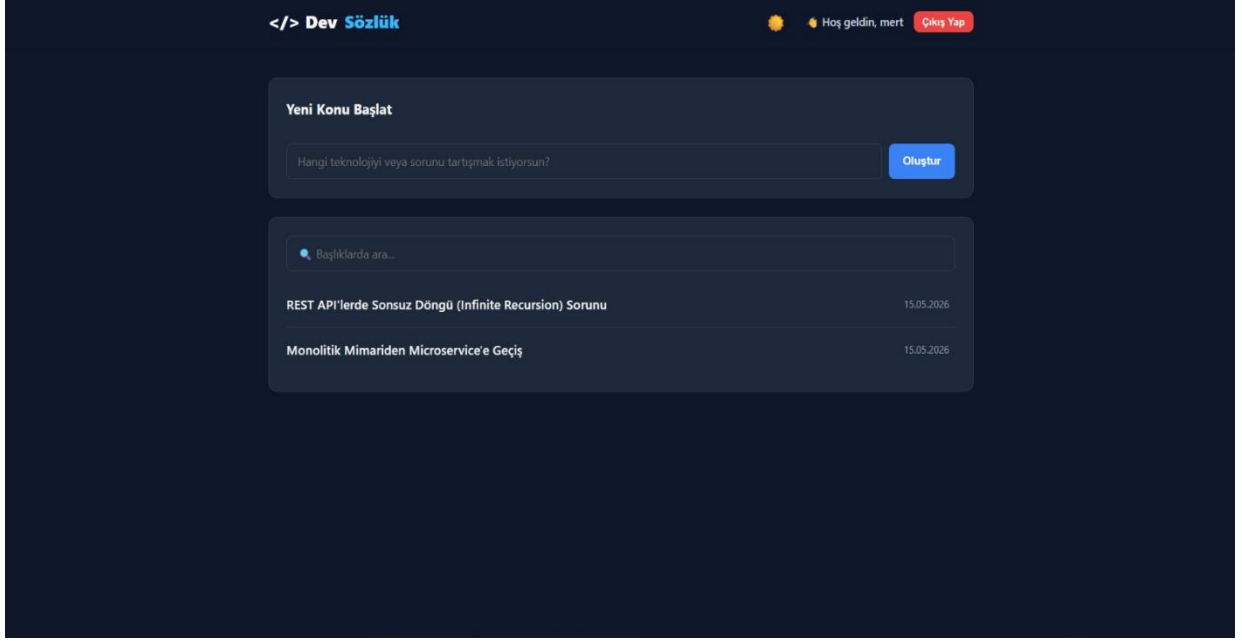
4.KULLANICI KILAVUZU



Giriş ve Kayıt Ekranı (Ziyaretçi Görünümü)

Bu ekran, sisteme henüz giriş yapmamış ziyaretçilerin karşılaştığı ana sayfadır.

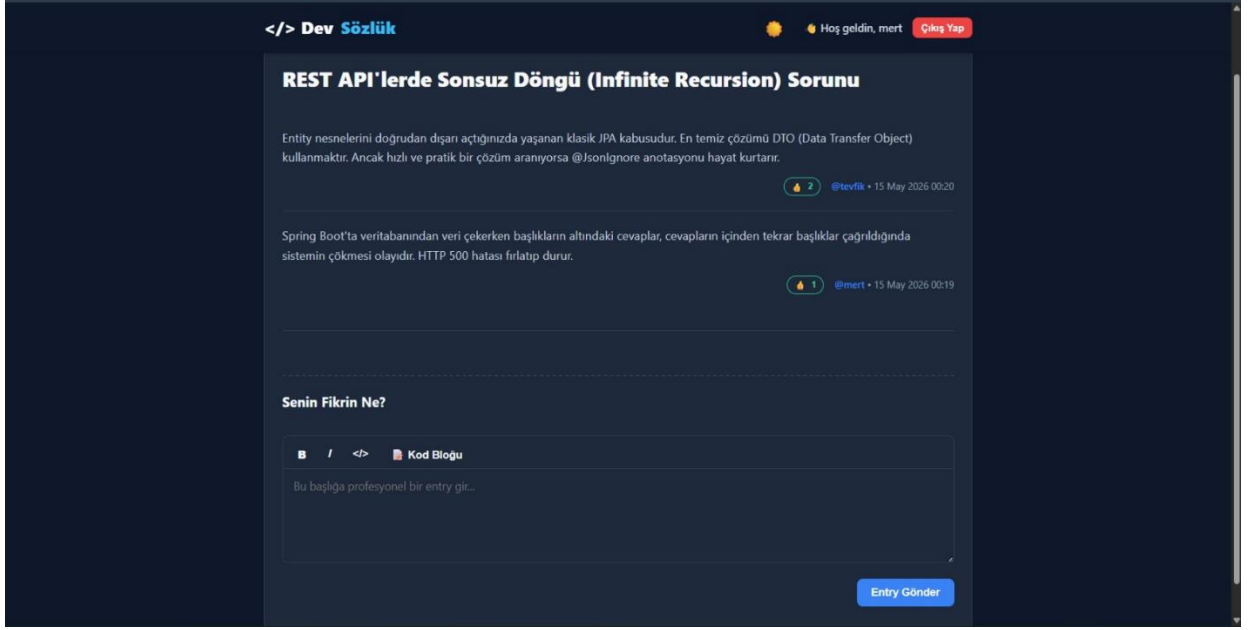
- **Ziyaretçi Ne Yapabilir?**
 - **Görüntüleme:** Sisteme üye olmadan veya giriş yapmadan mevcut açılmış başlıkları (örneğin: "REST API'lerde Sonsuz Döngü Sorunu") ve tarihlerini listeleyebilir.
 - **Arama:** Arama çubuğunu ("Başlıklarda ara...") kullanarak mevcut konular arasında arama yapabilir.
 - **Giriş/Kayıt:** İşlem yapabilmek için açılır panel üzerinden kullanıcı adı ve şifresi ile "Giriş Yap" butonunu kullanabilir veya hesabı yoksa "Yeni Kayıt Ol" butonuna tıklayarak sisteme dahil olabilir.



Kullanıcı Ana Sayfası (Standart Kullanıcı Görünümü)

Kullanıcı (örnekte "mert" adlı kullanıcı) başarılı bir şekilde giriş yaptıktan sonra bu ekrana yönlendirilir. Sağ üst köşede kullanıcının adı ve "Çıkış Yap" butonu belirir.

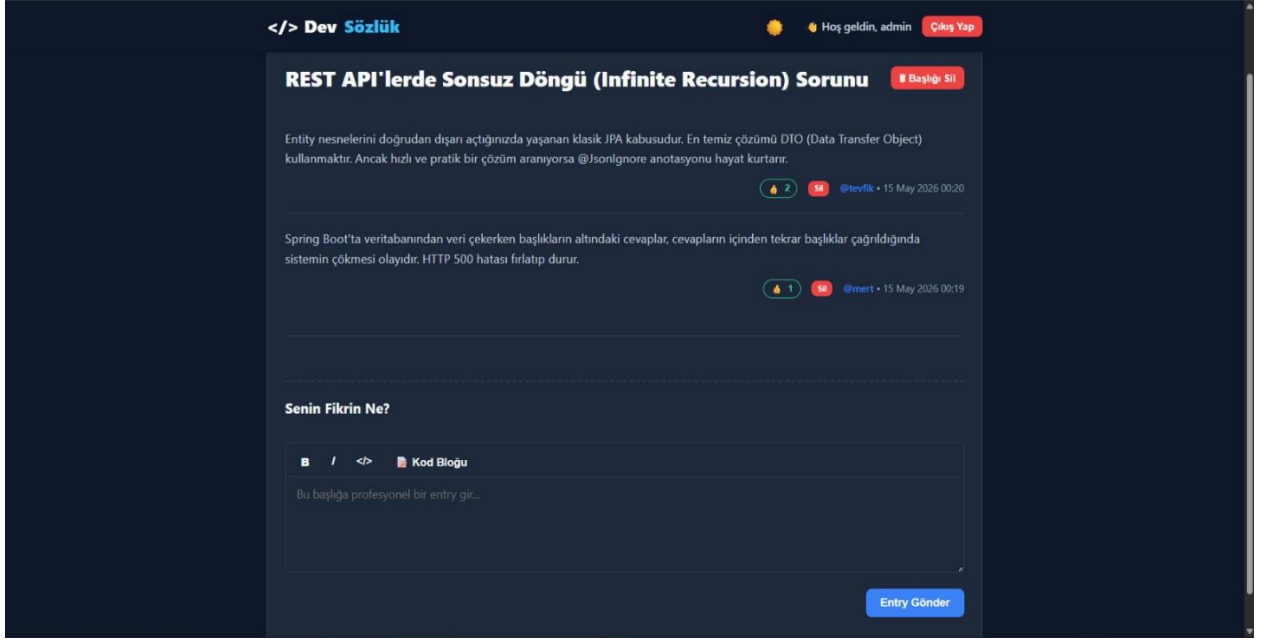
- **Standart Kullanıcı Ne Yapabilir?**
 - **Yeni Konu Açma:** En üstte yer alan "Yeni Konu Başlat" bölümündeki metin kutusuna tartışmak istediği konuyu veya sorunu yazıp "Oluştur" butonuna basarak sözlükte yeni bir başlık açabilir.
 - **Navigasyon:** Ziyaretçiler gibi mevcut başlıkları listeleyebilir, arama yapabilir ve ilgisini çeken bir başlığa tıklayarak detay sayfasına gidebilir.
 - **Oturum Yönetimi:** İşlemlerini bitirdiğinde sağ üstteki "Çıkış Yap" butonu ile oturumunu güvenli şekilde kapatabilir.



Konu Detay ve Yorum Ekranı (Standart Kullanıcı Görünümü)

Bir başlığa tıklandığında açılan sayfadır. Bu sayfada konu hakkındaki tartışmalar ve gönderiler (entry'ler) yer alır.

- **Standart Kullanıcı Ne Yapabilir?**
 - **Entry (İçerik) Okuma:** Başlık altında diğer kullanıcıların (örneğin "tevfik" ve "mert") girdiği içerikleri, yazılma tarihleri ve saatleri ile birlikte okuyabilir.
 - **Etkileşim (Beğenme):** Beğendiği entry'lerin altındaki beğeni butonuna basarak etkileşimde bulunabilir. Butonun yanındaki sayı, o gönderinin aldığı toplam beğeni sayısını gösterir.
 - **Yeni Entry Girme:** Sayfanın en altındaki "Senin Fikrin Ne?" bölümünü kullanarak konuya kendi yorumunu ekleyebilir. Metin editörü sayesinde yazısını kalın (B), italik (I) yapabilir veya kod blokları (</>, Kod Bloğu) ekleyerek teknik terimleri daha okunaklı hale getirebilir. Sonrasında "Entry Gönder" butonu ile yorumunu yayınlar.



Yönetici (Admin) Ekranı

Bu ekran, sisteme üst düzey yetkilerle donatılmış bir yönetici (örnekte "admin") giriş yaptığında açılan konu detay sayfasıdır. Standart kullanıcının gördüğü her şeyi görür, ancak ek denetim (moderasyon) yetkilerine sahiptir.

- **Yönetici (Admin) Ne Yapabilir?**
 - **Başlık Silme:** Sayfanın sağ üst köşesinde, başlığın hemen yanında bulunan kırmızı "Başlığı Sil" butonuna tıklayarak ilgili konuyu ve altındaki tüm entry'leri sistemden tamamen kaldırabilir.
 - **Entry (Yorum) Silme:** Kurallara uymayan veya hatalı olan bireysel gönderileri, her entry'nin sağ alt köşesinde (beğeni butonunun yanında) beliren kırmızı "Sil" butonunu kullanarak silebilir.
 - **Diğer Standart İşlemler:** Kendi hesabı üzerinden standart kullanıcıların yaptığı gibi entry girebilir, beğeni yapabilir ve çıkış işlemlerini gerçekleştirebilir.

5. KURULUM TALİMATLARI

Bu proje, Spring Boot (Backend) ve Vanilla JS/HTML/CSS (Frontend) kullanılarak "Monolitik" yapıda geliştirilmiştir. Frontend ve backend aynı sunucu üzerinden çalışmaktadır.

Gereksinimler

Projeyi kendi bilgisayarınızda çalıştırabilmek için sisteminizde aşağıdakilerin yüklü olması gerekmektedir:

Java Development Kit (JDK): Versiyon 17 veya üzeri.

Git: Projeyi bilgisayarınıza indirmek için.

PostgreSQL: Veritabanı yönetimi için (Yerel veya Neon.tech gibi bulut tabanlı bir hizmet kullanılabilir).

IDE (İsteğe Bağlı): VS Code, IntelliJ IDEA veya Eclipse.

(Not: Frontend backend'e entegre olduğu için Node.js yüklemenize gerek yoktur.)

Adım 1: Repository'yi Clone Etme

Terminalinizi (veya komut satırınızı) açın ve projeyi bilgisayarınıza indirin:

Bash

```
git clone https://github.com/tevfikgorel5/devsozluk.git
cd devsozluk
```

Adım 2: Veritabanı Ayarları

Projenin veritabanı ile iletişim kurabilmesi için application.properties dosyasını ayarlamanız gerekmektedir.

PostgreSQL üzerinde devsozluk adında boş bir veritabanı oluşturun (veya Neon.tech üzerinden bir URL alın).

Proje dizininde src/main/resources/application.properties dosyasını açın.

İçeriği kendi veritabanı bilgilerinize göre aşağıdaki gibi güncelleyin:

Properties

Veritabanı Bağlantı Ayarları

spring.datasource.url=jdbc:postgresql://localhost:5432/devsozluk

```
spring.datasource.username=veritabani_kullanici_adiniz
spring.datasource.password=veritabani_sifreniz
# Hibernate Ayarları (Tabloları otomatik oluşturur)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Adım 3: Bağımlılıkları Yükleme

Proje bağımlılıklarını (kütüphaneleri) indirmek için Gradle wrapper kullanıyoruz. Terminalde proje ana dizininde (içinde gradlew dosyası olan kasörde) şu komutu çalıştırın:

Mac / Linux ortamı için:

Bash

```
./gradlew build
```

Windows ortamı için:

DOS

```
gradlew.bat build
```

(Bu komut, Spring Boot ve veritabanı sürücüleri gibi tüm gerekli paketleri otomatik olarak indirecektir.)

Adım 4: Projeyi Çalıştırma (Backend & Frontend)

Her şey hazır! Şimdi sunucuyu ayağa kaldırabiliriz. Yine proje ana dizininde şu komutu çalıştırın:

Mac / Linux ortamı için:

Bash

```
./gradlew bootRun
```

Windows ortamı için:

DOS

```
gradlew.bat bootRun
```

Terminalde %80 Started DevsozlukApplication in X seconds yazısını gördüğünüzde sunucu başarıyla çalışmış demektir.

Adım 5: Tarayıcıdan Erişme

Frontend dosyalarımız Spring Boot tarafından sunulmaktadır. Projeyi kullanmaya başlamak için favori web tarayıcınızı açın ve şu adrese gidin:

<http://localhost:8080>

Test Etmek İçin: Sağ üst köşedeki "Kayıt Ol" butonuna tıklayarak yeni bir kullanıcı oluşturun ve ardından giriş yapın. Eğer başarıyla giriş yapabiliyorsanız, backend, frontend ve veritabanı kusursuz bir şekilde birbiriyle haberleşiyor demektir.

6. GÖREV DAĞILIMI

Proje geliştirme süreci, iş yükünü dengelemek ve uzmanlık alanlarına odaklanmak amacıyla "Backend (Arka Yüz)" ve "Frontend (Ön Yüz)" olmak üzere iki ana ekibe ayrılmıştır. Her bir ekip üyesinin proje aşamalarındaki temel katkıları aşağıda şeffaf bir şekilde listelenmiştir.

Backend Ekibi (Java Spring Boot & PostgreSQL)

1. Mert Sefa Taş (Backend Geliştirici)

Analiz Raporu: Veritabanı gereksinimlerinin belirlenmesi ve REST API uç noktalarının (endpoints) listelenip planlanması.

Tasarım Raporu: Sınıf diyagramlarının (UML) oluşturulması ve Spring Boot monolitik mimarisinin tasarlanması.

İmplementasyon: CevapController ve KullaniciController sınıflarının kodlanması. İstemciden (Frontend) gelen hatalı kimlik verilerinden kaynaklanan "400 Bad Request" (Katı Veri Tipi) hatalarının çözülmesi ve sistemin kurşun geçirmez hale getirilmesi. Oylama algoritmasının API bağlantılarının yazılması.

Final Raporu: Kurulum talimatlarının (Gereksinimler, Build ve Run komutları) yazılması ve "Karşılaşılan Zorluklar ve Çözümleri (Troubleshooting)" bölümünün dokümantasyonu.

2. Hamza Ergüven (Backend Geliştirici)

Analiz Raporu: Sistem güvenliği (şifreleme) ihtiyaçlarının araştırılması ve veri akış diyagramlarının çıkarılması.

Tasarım Raporu: Veritabanı Varlık-İlişki (ER) diyagramlarının çizilmesi (Kullanıcı, Baslık ve Cevap tabloları arası One-to-Many / Many-to-One ilişkiler).

İmplementasyon: Veritabanı tablolarının JPA/Hibernate kullanılarak ayağa kaldırılması. MessageDigest kullanılarak SHA-256 şifre hashleme algoritmasının sisteme entegre edilmesi. REST API'de karşılaşılan Infinite Recursion (Sonsuz Döngü) hatasının @JsonIgnore kullanılarak çözülmesi. Neon.tech veritabanında eksik olan beğeni_sayisi kolonunun SQL şemasına eklenip "500 Internal Server Error" çökmesinin engellenmesi.

Final Raporu: "Yaptıklarımız, Yapmadıklarımız ve Nedenleri" bölümünün (vazgeçilen Spring Security ve framework kararlarının) dürüst ve teknik bir dille rapora aktarılması.

Frontend Ekibi (Vanilla JS, HTML5, CSS3)

3. Sefa Koyuncu (Frontend Geliştirici)

Analiz Raporu: Kullanıcı arayüzü (UI) senaryolarının (Use Cases) belirlenmesi ve ekran gereksinimlerinin (Kayıt, Ana Sayfa, Başlık Detay) çıkarılması.

Tasarım Raporu: Arayüz taslaklarının (Wireframes) oluşturulması, renk paletinin seçilmesi ve CSS değişkenleri (CSS Variables) kullanılarak modern Açık/Koyu (Light/Dark Mode) tema mimarisinin tasarlanması.

İmplementasyon: HTML/CSS iskeletinin responsive (uyumlu) bir şekilde kodlanması. Kullanıcı deneyimini artırmak için API'den veri beklenirken gösterilen "Skeleton Loader" (animasyonlu yükleniyor çubuğu) yapısının geliştirilmesi. Kendi iç Markdown parser (düzenleyici) yapımızın Regex (Düzenli İfadeler) ile yazılarak sisteme eklenmesi.

Final Raporu: Proje özeti (Abstract) ve hedefler bölümünün yazılması, arayüz ekran görüntülerinin (UI Screenshots) raporun eklerine düzenli bir şekilde yerleştirilmesi.

4. Tefvik Görel (Frontend Geliştirici)

Analiz Raporu: Frontend ile Backend arasındaki veri iletişim (JSON) formatlarının, istek (Request) ve yanıt (Response) modellerinin planlanması.

Tasarım Raporu: Dinamik DOM manipölasyonu (sayfa yenilenmeden verilerin güncellenmesi) mantığının ve JavaScript fonksiyon akış diyagramlarının tasarlanması.

İmplementasyon: Vanilla JavaScript ve asenkron Fetch API kullanılarak arka yüz haberleşmesinin (kayıt, giriş, listeleme) sağlanması. Oylama (beğeni) butonlarının API ile bağlanarak sayfa yenilenmeden dinamik sayaç güncellenmesinin kodlanması. Hatalı dönen veya silinen kullanıcı verilerinin JavaScript'te sebep olduğu "Sessiz Çökme" (Silent Crash) hatalarının Try-Catch bloklarıyla çözülmesi.

Final Raporu: Kullanılan teknolojiler ve mimari yapı (Tech Stack) bölümünün yazılması, projenin genel sayfa düzeninin (Formatting) yapılarak teslim formatına uygun hale getirilmesi.

7. Yaptıklarımız, Yapmadıklarımız ve Nedenleri

Proje geliştirme sürecinde, başlangıçta planlanan analiz ve tasarım dokümanlarına büyük ölçüde sadık kalınmış olsa da, teknik kısıtlar, zaman yönetimi ve sistem stabilitesi göz önüne alınarak bazı revizyonlara gidilmiştir.

7.1. Yapılan ve Yapılamayan Gereksinimler

Yaptıklarımız: Başarılı bir şekilde implemente edilen temel gereksinimler; kullanıcı kayıt ve giriş işlemleri, veritabanında şifre hashleme, dinamik konu başlığı oluşturma, başlıklara Markdown destekli entry girme, asenkron oylama (beğeni) sistemi ve Admin rolüyle içerik silebilme özellikleridir.

Yapamadıklarımız: Başlangıçta planlanan "Detaylı Kullanıcı Profili Sayfası", "Entry'lere Yorum Yapma (İç İç Cevaplar)" ve "Şifremi Unuttum (E-posta aktivasyonu)" özellikleri projeye dahil edilememiştir.

Nedeni: Proje teslim süresinin kısıtlı olması nedeniyle, efor ağırlıklı olarak projenin "çekirdek (core)" işlevlerine (başlık/entry akışı ve oylama algoritması) kaydırılmıştır. Hatalı çalışan çok sayıda özellik sunmak yerine, az ama kusursuz ve çökmeden çalışan bir sistem teslim etmek tercih edilmiştir.

7.2. Tasarımda ve Sınıflarda Değişen Kısımlar

Sayfalama (Pagination) Mimarisinin İptali: Tasarım aşamasında CevapController üzerinden entry'lerin sayfa sayfa (Page<Cevap>) çekilmesi planlanmış ve kodlanmıştı. Ancak projeye sonradan eklenen "En çok beğeni alan entry'yi en üste çıkarma" özelliği ile sayfalama yapısı JavaScript tarafında senkronizasyon (sessiz çökme) sorunları yarattı. Bu nedenle Page mimarisinden vazgeçilerek, verilerin OrderByBegeniSayisiDesc sorgusu ile düz bir Liste (List<Cevap>) olarak dönülmesi kararlaştırıldı.

Sonsuz Döngü (Infinite Recursion) Müdahalesi: Cevap ve Baslik Entity sınıflarındaki çift yönlü ilişkiler (Bidirectional Mapping), REST API üzerinden JSON'a dönüştürülürken sonsuz döngü hatasına (StackOverflowError) sebep oldu. Tasarıma sonradan müdahale edilerek ilgili alanlara @JsonIgnore anotasyonu eklenip döngü kırıldı.

7.3. Vazgeçilen Teknolojiler ve Kütüphaneler

Spring Security: Bařlangıçta kimlik doęrulama ve yetkilendirme işlemleri için Spring Security kullanılması planlanmıştı. Ancak bu kütüphanenin yüksek öğrenme eğrisi ve ağır konfigürasyon gereksinimleri, projenin esnekliğini yavaşlattı. Neden Vazgeçildi: Zamanı verimli kullanmak adına projeden çıkarıldı. Yerine, Java'nın kendi içinde yerleşik bulunan MessageDigest sınıfı kullanılarak SHA-256 şifreleme algoritması manuel olarak entegre edildi. Böylece sisteme dışarıdan ağır bir bağımlılık eklenmeden yüksek güvenlik sağlandı.

React.js / Vue.js (Ayrı Frontend Sunucusu): Ön yüz tasarımı için modern bir framework kullanmak hedeflenmişti. Ancak sistemi Backend ve Frontend olarak iki ayrı portta ayaęa kaldırmak CORS hatalarına ve dağıtım (deployment) zorluklarına yol açtı. Neden Vazgeçildi: Monolitik yapıya sadık kalmak adına Vanilla JavaScript, HTML ve CSS tercih edildi; DOM manipölasyonları için asenkron Fetch API kullanılarak Single Page Application (SPA) hissiyatı başarılı bir şekilde verildi.

Hazır Markdown (External Library) Kütüphaneleri: Kullanıcıların kalın, italik veya kod bloęu yazabilmesi için dışarıdan hazır bir kütüphane entegre etmek yerine, sistemin hafifliğini korumak amacıyla kendi JavaScript tabanlı, Regex (Düzenli İfadeler) kullanan parseMarkdown fonksiyonumuz yazılmıştır.